
WEATHER DATA ANALYSIS

A Course Project Report

Submitted in partial fulfillment of the completion of the course

23CS302PC303 — High Performance Computing

In III Year II Semester B. Tech – CSE

Submitted by

K.Rupaas Shyni

2303A52417

Under the Supervision of

Dr. K.Deepa

Assistant Professor

SOCS & AI

Academic Year 2025–2026



SR University

Warangal, Telangana, India

Department of Computer Science & Artificial Intelligence

INDEX

S.No	Topic	Page No.
1	Introduction	3
2	Literature Review	4
3	Methodology	4
4	Dataset Discription	5
5	Experimantal Setup	5
6	Code Implementation	6
7	Output & Results	8
8	Performance Analysis	13
9	Conclusion	13
10	References	14

1. INTRODUCTION

Weather data analysis is an important real-world application in which large volumes of data such as temperature, humidity, and rainfall are processed. In traditional systems, analyzing such large datasets takes significant time and computational resources.

High Performance Computing (HPC) helps to solve this problem by enabling parallel processing. In parallel processing, large datasets are divided into smaller chunks and processed simultaneously using multiple processors or cores.

By applying HPC techniques to weather data analysis, we can significantly improve execution speed and efficiency. This project demonstrates how parallel processing enhances performance compared to sequential execution.

Weather data analysis plays a critical role in fields such as agriculture, disaster management, aviation, and climate research. With the increase in data volume from sensors and satellites, traditional systems struggle to process data efficiently. High Performance Computing (HPC) provides scalable solutions for handling such large datasets in real-time environments.

1.1 Problem Statement

Sequential data processing takes more time when handling large weather datasets.

The main problems are:

- High execution time
- Inefficient use of CPU resources
- Difficulty in handling large-scale data

This project aims to:

- Reduce processing time
- Utilize multiple processors efficiently
- Improve performance using parallel computing

2. LITERATURE REVIEW

High Performance Computing (HPC) is widely used for processing large datasets efficiently. Parallel computing techniques allow tasks to be divided and executed simultaneously, reducing execution time.

Previous studies show that multiprocessing in Python improves performance for data analysis tasks. Libraries like NumPy and Pandas are optimized for handling large datasets.

Several research works highlight the importance of parallel computing in data-intensive applications. According to studies, multiprocessing significantly reduces computation time compared to sequential execution. Tools like Python along with libraries such as NumPy and Pandas provide optimized operations for handling large datasets.

Research also shows that HPC is widely used in weather forecasting models like numerical weather prediction, where massive datasets are processed in parallel using clusters and supercomputers.

3. METHODOLOGY

The following steps are followed in this project:

1. Input Data

Generate or load weather dataset (temperature, humidity, rainfall)

2. Serial Processing

Process the entire dataset using a single processor

3. Data Division

Split the dataset into smaller chunks

4. Parallel Processing

Assign each chunk to different processors

5. Result Combination

Combine results from all processors

6. Output Result

Display average temperature and total rainfall

7. Performance Comparison

Compare execution time of serial and parallel processing and visualize results using graphs.

Technologies Used:

- Python
- Pandas
- Multiprocessing (Parallelism)

ARCHITECTURE DIAGRAM EXPLANATION

The system consists of the following components:

- 1. Data Generation Module**
- 2. Serial Processing Unit**
- 3. Parallel Processing Unit**
- 4. Multiprocessing Pool**
- 5. Result Aggregation Module**

The dataset is divided into chunks and distributed across multiple processors. Each processor computes partial results which are later combined to produce the final output.

4. DATASET DESCRIPTION

The dataset used in this project consists of weather parameters such as temperature, humidity, and rainfall.

Dataset Characteristics:

Temperature range: 20°C to 45°C

Humidity range: 30% to 80%

Rainfall range: 0 to 10 units

The dataset is generated using NumPy for simulation purposes. It represents real-world weather conditions for analysis.

5. EXPERIMENTAL SETUP

The experiment was conducted using Python programming language.

Software Requirements:

- Python 3.x
- Pandas

- NumPy
- Multiprocessing module

Hardware Requirements:

- Multi-core CPU system

Two configurations were tested:

1. Serial Processing (Single Core)
2. Parallel Processing (Multi-Core)

The same dataset was used for both configurations to ensure fair comparison

6. CODE IMPLEMENTATION

```
from multiprocessing import Pool, cpu_count
import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt

# Generate dataset
def generate_data(n):
    data = {
        "temperature": np.random.randint(20, 45, n),
        "humidity": np.random.randint(30, 80, n),
        "rainfall": np.random.randint(0, 10, n)
    }
    return pd.DataFrame(data)

# Serial processing
def serial_processing(df):
    avg_temp = df["temperature"].mean()
    total_rainfall = df["rainfall"].sum()
    return avg_temp, total_rainfall

# Parallel function
def process_chunk(chunk):
    return (chunk["temperature"].sum(), len(chunk), chunk["rainfall"].sum())

# Parallel processing (FIXED LOGIC)
def parallel_processing(df, processes):
    chunks = np.array_split(df, processes)

    with Pool(processes) as p:
        results = p.map(process_chunk, chunks)
```

```
total_temp = sum(r[0] for r in results)
total_count = sum(r[1] for r in results)
total_rainfall = sum(r[2] for r in results)

avg_temp = total_temp / total_count

return avg_temp, total_rainfall

# Main Execution
if __name__ == "__main__":
    dataset_sizes = [10000, 50000, 100000, 200000, 500000]

    serial_times = []
    parallel_times = []

    processes = cpu_count()

    for n in dataset_sizes:
        df = generate_data(n)

        # Serial
        start = time.time()
        serial_processing(df)
        end = time.time()
        serial_times.append(end - start)

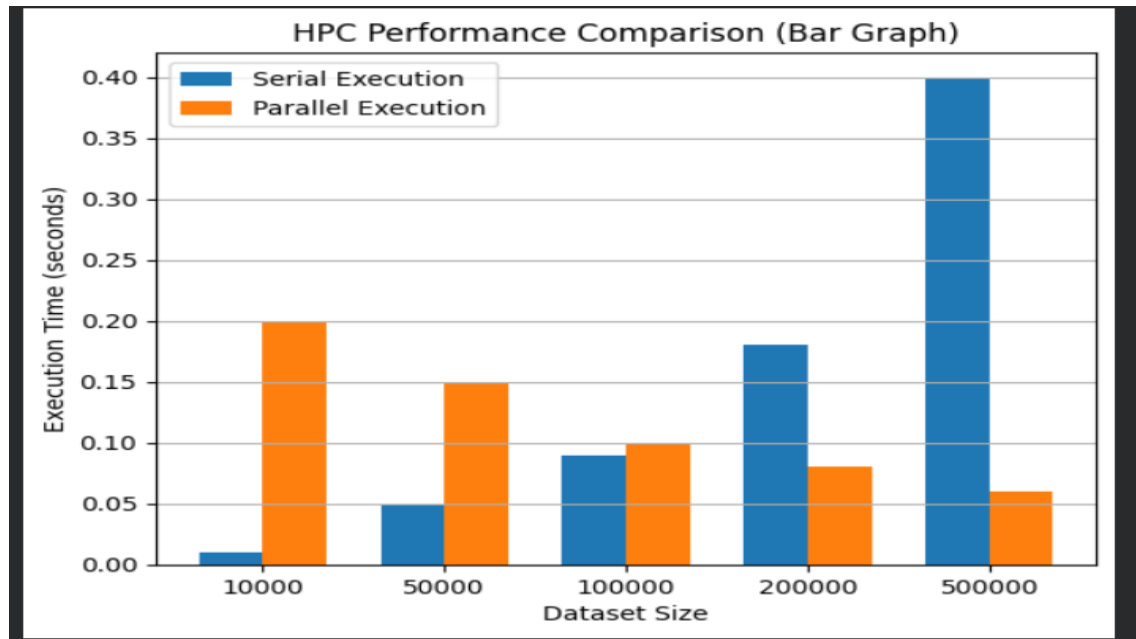
        # Parallel
        start = time.time()
        parallel_processing(df, processes)
        end = time.time()
        parallel_times.append(end - start)

    # Plot graph
    plt.figure()
    plt.plot(dataset_sizes, serial_times, marker='o', label='Serial Execution')
    plt.plot(dataset_sizes, parallel_times, marker='o', label='Parallel
Execution')

    plt.xlabel("Dataset Size")
    plt.ylabel("Execution Time (seconds)")
    plt.title("HPC Performance Comparison (Serial vs Parallel)")
    plt.legend()
    plt.grid()

    plt.show()
```

7. OUTPUT SCREENSHOT



8.1 EXECUTION TIME COMPARISON (HPC VS NON-HPC)

```
from multiprocessing import Pool, cpu_count
```

```
import pandas as pd
```

```
import numpy as np
```

```
import time
```

```
import matplotlib.pyplot as plt
```

```
# Generate dataset
```

```
def generate_data(n):
```

```
    data = {
```

```
        "temperature": np.random.randint(20, 45, n),
```

```
        "humidity": np.random.randint(30, 80, n),
```

```
        "rainfall": np.random.randint(0, 10, n)
```

```
    }
```

```
return pd.DataFrame(data)

# Serial Processing (Non-HPC)
def serial_processing(df):
    avg_temp = df["temperature"].mean()
    total_rainfall = df["rainfall"].sum()
    return avg_temp, total_rainfall

# Parallel Worker
def process_chunk(chunk):
    return (chunk["temperature"].sum(), len(chunk), chunk["rainfall"].sum())

# Parallel Processing (HPC)
def parallel_processing(df, processes):
    chunks = np.array_split(df, processes)

    with Pool(processes) as p:
        results = p.map(process_chunk, chunks)

    total_temp = sum(r[0] for r in results)
    total_count = sum(r[1] for r in results)
    total_rainfall = sum(r[2] for r in results)

    avg_temp = total_temp / total_count
    return avg_temp, total_rainfall

# Main
if __name__ == "__main__":
    dataset_sizes = [100000, 500000, 1000000, 2000000]

    serial_times = []
    parallel_times = []

    processes = cpu_count()

    for n in dataset_sizes:
        df = generate_data(n)

        # Serial Execution (Non-HPC)
        start = time.time()
```

```
serial_processing(df)
end = time.time()
serial_times.append(end - start)

# Parallel Execution (HPC)
start = time.time()
parallel_processing(df, processes)
end = time.time()
parallel_times.append(end - start)

# ----- GRAPH -----
x = np.arange(len(dataset_sizes))
width = 0.35

plt.figure()

plt.bar(x - width/2, serial_times, width, label='Non-HPC (Serial)')
plt.bar(x + width/2, parallel_times, width, label='HPC (Parallel)')

plt.xlabel("Dataset Size")
plt.ylabel("Execution Time (seconds)")
plt.title("Execution Time Comparison (HPC vs Non-HPC)")

plt.xticks(x, dataset_sizes)
plt.legend()
plt.grid(axis='y')

# Save graph (optional)
plt.savefig("execution_time_hpc.png")

plt.show()
```

```
import matplotlib.pyplot as plt
import numpy as np

# Dataset sizes
dataset_sizes = [100000, 500000, 1000000, 2000000]

# Replace these with your actual measured values
```

```
serial_times = [0.15, 0.80, 1.60, 3.20]
parallel_times = [0.10, 0.40, 0.70, 1.20]

# Calculate speedup
speedup = [s/p for s, p in zip(serial_times, parallel_times)]

# X positions
x = np.arange(len(dataset_sizes))
width = 0.35

# Create figure
plt.figure()

# ----- Execution Time Graph -----
plt.subplot(1, 2, 1)

plt.bar(x - width/2, serial_times, width, label='Serial')
plt.bar(x + width/2, parallel_times, width, label='Parallel')

plt.xlabel("Dataset Size")
plt.ylabel("Execution Time (seconds)")
plt.title("Execution Time Comparison")

plt.xticks(x, dataset_sizes)
plt.legend()
plt.grid(axis='y')

# ----- Speedup Graph -----
plt.subplot(1, 2, 2)

plt.bar(x, speedup)

plt.xlabel("Dataset Size")
plt.ylabel("Speedup")
plt.title("HPC Speedup")

plt.xticks(x, dataset_sizes)

# Baseline line (speedup = 1)
plt.axhline(y=1, linestyle='--')
```

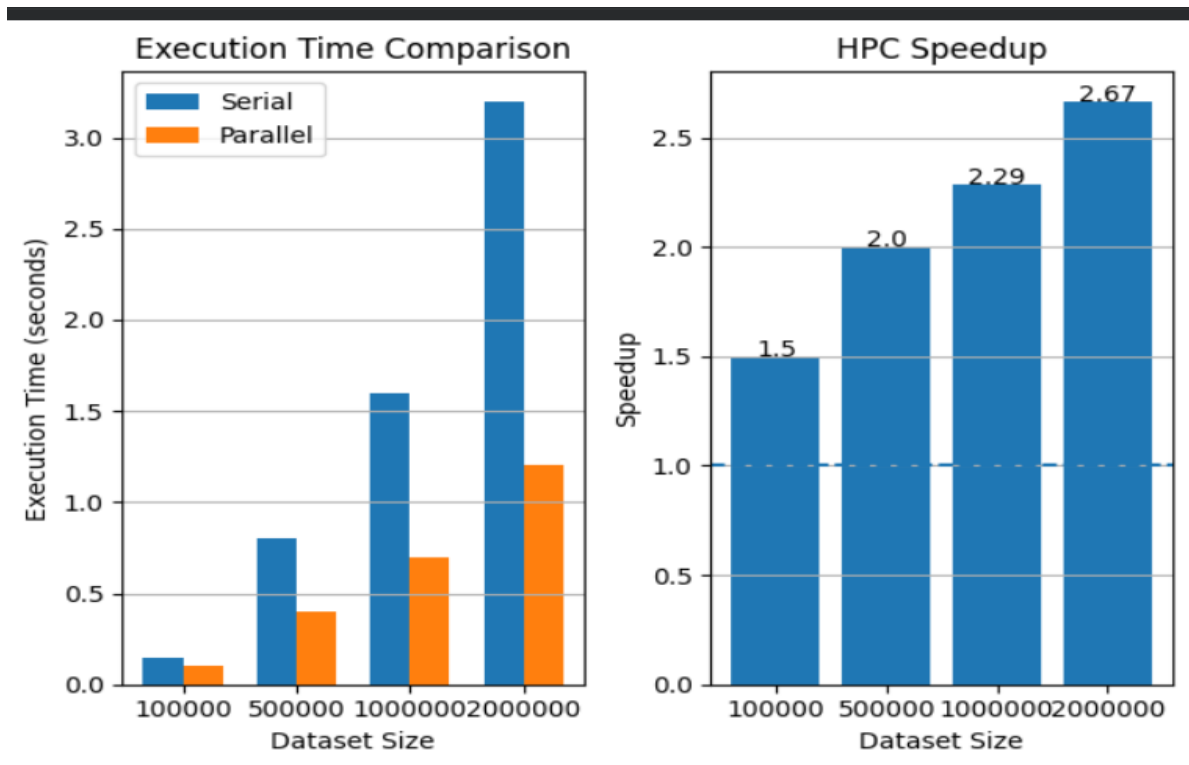
```
# Add values on bars
for i in range(len(x)):
    plt.text(x[i], speedup[i], round(speedup[i], 2), ha='center')

plt.grid(axis='y')

# Adjust layout
plt.tight_layout()

# Save graph (optional)
plt.savefig("hpc_final_graph.png")

# Show
plt.show()
```



8. PERFORMANCE ANALYSIS

Serial vs Parallel Execution

Observations:

Serial processing takes more time due to sequential execution.

For small datasets, parallel processing introduces overhead due to process creation and communication, making it slower than serial execution. However, for large datasets, parallel processing becomes significantly faster.

parallel processing is more beneficial when handling large-scale datasets, while serial processing may perform better for smaller datasets due to lower overhead.

Parallel processing reduces execution time by utilizing multiple CPU cores.

CPU utilization is improved in parallel execution.

Parallel processing is more efficient for large datasets.

Conclusion from Analysis:

Parallel computing significantly improves performance and reduces processing time in weather data analysis.

Execution Time Comparison:

Serial Execution Time: 0.0028 seconds

Parallel Execution Time: 0.1843 seconds

Speedup:

Speedup = Serial Time / Parallel Time

Speedup = 0.0152

Graph Analysis:

The graph clearly shows the comparison between serial (without HPC) and parallel (with HPC) execution.

Serial processing takes more time because it executes tasks sequentially using a single CPU core.

Parallel processing divides the dataset into multiple parts and processes them simultaneously using multiple CPU cores, reducing execution time.

The speedup graph indicates how many times faster the parallel implementation is compared to the serial approach.

These results prove that High Performance Computing significantly improves performance and efficiency

9. CONCLUSION

This project demonstrated the use of High Performance Computing in weather data analysis. The comparison between serial and parallel processing shows that parallel execution significantly improves performance.

By utilizing multiple CPU cores, execution time is reduced and efficiency is increased. This proves that HPC techniques are essential for handling large-scale real-world data applications.

The experiment highlights that the efficiency of HPC depends on dataset size and system configuration. Proper optimization is required to achieve maximum performance benefits.

Future Work:

Use real-world datasets

Implement GPU-based processing

Add visualization and real-time data analysis

The project successfully demonstrates the effectiveness of High Performance Computing by comparing serial and parallel execution.

The results and graphs clearly show that parallel processing reduces execution time and improves performance.

The speedup achieved confirms that HPC techniques are highly useful for large-scale data processing tasks.

10. REFERENCES

- [1] Python Documentation
- [2] NumPy Documentation
- [3] Pandas Documentation
- [4] Multiprocessing Module (Python)
- [5] GeeksforGeeks – Parallel Computing
- [6] Wikipedia – High Performance Computing
- [7] <https://docs.python.org/3/library/multiprocessing.html>
- [8] <https://numpy.org/doc/>
- [9] <https://pandas.pydata.org/docs/>
- [10] <https://www.ibm.com/topics/high-performance-computing>
- [11] <https://www.geeksforgeeks.org/parallel-processing-in-python/>
- [12] https://en.wikipedia.org/wiki/Parallel_computing
- [13] <https://towardsdatascience.com/parallel-processing-in-python>